part03_cli — Development Environment and adk CLI

Run from the `part03_cli` directory (= parent directory of the agents). This corresponds to lecture notes **Part 3 · 03. Development Environment and CLI** (Slides 30–48).

```
part03_cli/
├── my_agent/             # 📁 Basic ADK agent scaffold (prompt-based Q&A)
├── my_agent_mod/         # 📁 Custom ADK agent (adds get_weather weather lookup tool)
├── my-agents-cli/        # 📁 Standard enterprise template from agents-cli (FastAPI, SSE, A2A,
Terraform)
├── my-agents-cli-mod/    # 📁 Custom agents-cli agent (real-time Open-Meteo weather & world time)
├── run_direct_runner.py  # 🔧 Direct Runner code: register → run → consume events (Slides 23–25 · 39)
├── adk_internals.py      # 🟢/🔧 Recreating the 5 steps that adk handles automatically (Slides 31–38)
├── adk_cli_map.py        # 🟢 Checking adk CLI source structure: command tree · AgentLoader · URI
scheme · route counts (Slides 32–36)
├── api_client.py         # 🔧 Invoking /run and /run_sse of adk api_server (Slide 43)
├── _path.py              # ⚙️ Parent directory and package path search helper
├── README-KR.md          # 📄 Lab guide in Korean
└── README-EN.md          # 📄 Lab guide in English (this file)
```

You only need one `.env` file at the lab root (`ADK_Labs/.env`). `adk` traverses up parent directories from the agent folder to locate it.

📂 Subdirectory Overview and Summary

Subdirectory	Tool / Framework	Core Details	Primary Run Commands
`my_agent`	`adk` CLI	`adk create my_agent` using Gemini's native LLM reasoning without external tools.	`adk run my_agent` `adk web --port 8000`
`my_agent_mod`	`adk` CLI	City weather lookup agent adding a local `get_weather` function tool (Function Calling) to the basic ADK scaffold.	`adk run my_agent_mod` `adk web --port 8000`
`my-agents-cli`	`google-agents-cli`	Enterprise-grade production template generated with `agents-cli create`. Includes FastAPI backend, SSE streaming, A2A protocol, Reasoning Engine adapter, automated tests (unit/integration/eval), and Terraform IaC.	`cd my-agents-cli` `agents-cli playground` `agents-cli install`
`my-agents-cli-mod`	`google-agents-cli`	Enhanced agent building upon `my-agents-cli` by modifying `app/agent.py` to query real-time external Open-Meteo REST APIs and IANA timezones worldwide.	`cd my-agents-cli-mod` `agents-cli playground`

1) `my_agent/` Summary

- **Structure**: `agent.py`, `__init__.py`, `.env`, `.gitignore`, `.adk/session.db`
- **Role**: Minimal baseline ADK agent (`gemini-3.5-flash`). Executes default system prompt instructions without extra tools.
- **Run**: From `part03_cli`: `adk run my_agent` or `adk web --port 8000`

2) `my_agent_mod/` Summary

- **Structure**: `agent.py`, `__init__.py`, `.env`, `.gitignore`, `.adk/session.db`
- **Role**: Weather-specialized agent (`weather_agent`) with local tool `get_weather(city: str)`.
- **Run**: From `part03_cli`: `adk run my_agent_mod` or `adk web --port 8000`

3) `my-agents-cli/` Summary

- **Structure**: `app/` (FastAPI, agent, A2A, Reasoning Engine), `tests/` (unit, integration, eval), `deployment/terraform/`, `pyproject.toml`, `Dockerfile`, etc.
- **Role**: Standard enterprise template equipped with mock tools (`get_weather` , `get_current_time`) and production features (A2A protocol, evaluation pipeline, Terraform infrastructure deployment).
- `uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000`

4) `my-agents-cli-mod/` Summary

- **Structure**: `my-agents-cli` architecture + `app/agent.py` real-world API integration + Web architecture guide slide deck
- **Role**: Real-time live weather and world clock queries via Open-Meteo API, complete with offline fallback (`AGENT_OFFLINE=1`).
- **Run**: Inside `my-agents-cli-mod/` : `agents-cli playground`

---

1. Same Agent, Two Execution Methods (Slides 23–25 · 31–39)

The backbone of an ADK application is the `Runner` . We run and compare the same `my_agent/agent.py` using two methods.

Method	What the Developer Writes	Hands-on Script
Direct Runner Code	root_agent + Runner · services · I/O handling code	`run_direct_runner.py`
adk CLI	root_agent (agent.py) only	`adk run my_agent` · `adk_internals.py`

① Direct Runner Code: Register → Run → Consume Events (🔑)

```
uv run run_direct_runner.py "Tell me the weather in Busan"
```

```
① Register   Runner(agent=weather_agent, session_service=InMemorySessionService)
② Execute    run_async(new_message='Tell me the weather in Busan')
   [Event 1] Model call → function_call get_weather({'city': 'Busan'})   (Model only requests)
   [Event 2] Tool execution → function_response get_weather: {'status': 'success', 'report': 'Overcast,
20°C'}   (Runner executes)
③ Consume   Final Response (Event 3): The current weather in Busan is overcast with a temperature of
20°C.
```

A single `run_async()` corresponds to one full cycle of the agent loop (Slide 25): when the model requests a `function_call`, the Runner invokes the tool and calls the model again with the output.

② What adk Handles on Your Behalf (🟢 ①~④ without API key, 🔑 ⑤)

Recreates the 5 steps that `adk run my_agent` performs internally using ADK's native code (`AgentLoader`, `load_dotenv_for_agent`, `create_*_service_from_options`).

```
uv run adk_internals.py                    # Steps ①~④ (No LLM calls)
uv run adk_internals.py --in_memory         # Selects services matching adk run --in_memory
uv run adk_internals.py --ask "Seoul weather?"   # Through step ⑤: one turn of adk run
```

```
① Discover       AgentLoader → LlmAgent name='weather_agent' (from my_agent/agent.py)
② Load .env      GOOGLE_API_KEY set=True · ADK_MODEL=gemini-3.5-flash-lite
③ Build Services session=PerAgentDatabaseSessionService → my_agent/.adk/session.db (SQLite, created on
1st session)
                 artifact=PerAgentFileArtifactService · memory=InMemoryMemoryService
④ Build Runner   Runner(app='my_agent', root_agent='weather_agent')
⑤ Provide Entry  In adk run, launches input('[user]: ') loop → runner.run_async()
```

- **The default session store is not InMemory.** Without extra flags, sessions are stored in the agent folder's `.adk/session.db` (SQLite). Specifying `--in_memory` or running in Cloud Run/Kubernetes defaults to InMemory. In production, use `--session_service_uri`.
- `adk web` and `adk api_server` build the exact same services and Runner; the only difference is the HTTP entry point ⑤ (Slide 38).
- `google.adk.cli.utils` is an internal ADK API that may change across versions (recreated here based on version 2.6.3 for learning).
- Running `adk run` or `adk_internals.py` creates the `my_agent/.adk/` folder, which can be safely deleted.

---

2. agents-cli (Slides 44–48, details in Part 13 Slides 143–156)

For an introduction to `agents-cli`, comparison with `adk`, command maps, project structures, and runtime mechanics, see `part13_agents_cli/`.

---

3. adk CLI Internal Structure (Slides 32–36)

Inspect the `google/adk/cli` source code (google-adk 2.14.0 = GitHub main) aligned with the lecture slides. Runs without an API key.

```
python adk_cli_map.py           # commands · loader · services · routes
```

Item	Slide	What to Check
`commands`	32	`adk` = `cli_tools_click.main` Click group, handler function per command
`loader`	34	`AgentLoader.load_agent('my_agent')` → `LlmAgent`
`services`	34	Session: `memory · sqlite · postgresql · mysql · agentengine`, Artifact: `memory · gs · file` ...
`routes`	35–36	api_server routes: 22 · adk web routes: 59 (`/dev/*` : 35)

---

4. Key adk Commands (Slides 40–43)

Command	Description
`adk create my_agent2`	Scaffolds agent.py · __init__.py · .env
`adk run my_agent`	Terminal REPL (e.g., `Tell me Seoul weather`)
`adk run --save_session --session_id s1 my_agent`	Saves `my_agent/s1.session.json` on exit → reuse for evaluation
`adk run --resume my_agent/s1.session.json my_agent`	Resumes conversation from a saved session
`adk run --replay queries.json my_agent`	`{"state": {...}, "queries": [...]}` , ut (not a session file)
`adk web --port 8000`	Developer Web UI: http://localhost:8000 (Events · Trace · State · Graph tabs)
`adk api_server --port 8000 --session_service_uri="sqlite:///sessions.db"`	Headless REST/SSE server

Flags may differ across versions; check `adk run --help` for the latest options.

---

5. Calling api_server

```
adk api_server --port 8000
```

In another terminal:

```
uv run api_client.py
```

Calling directly with curl:

```
# Create session
curl -X POST localhost:8000/apps/my_agent/users/u1/sessions/s1

# Query agent
curl -X POST localhost:8000/run -H "Content-Type: application/json" -d "
{\"app_name\":\"my_agent\",\"user_id\":\"u1\",\"session_id\":\"s1\",\"new_message\":
{\"role\":\"user\",\"parts\":[{\"text\":\"Seoul weather?\"}]}}"
```